

BOUND: Boundary-Oriented Model Editing for Package Hallucination

Abstract—Large language models (LLMs) have demonstrated strong capabilities in software engineering tasks, such as code generation, library recommendation, and dependency configuration. However, recent studies show that LLMs may suffer from package hallucination, where they generate non-existent or invalid package names. These hallucinations can be exploited in software supply chain attacks, as attackers may register malicious packages under hallucinated names. Therefore, mitigating package hallucination is important for improving the reliability and security of LLM-assisted software development. In this paper, we introduce BOUND, a lightweight localized model editing framework for mitigating package hallucinations in LLMs. BOUND formulates package hallucination mitigation as a package-validity boundary editing problem, where the boundary refers to the model’s ability to distinguish valid packages from hallucinated package names under a given task context. It first locates modules related to package hallucination through a risk-aware localization strategy, and then edits these modules with lightweight LoRA adapters using a boundary-aware objective that reinforces valid packages, suppresses hallucinated packages, and preserves locality behavior. Experimental results show that BOUND effectively reduces package hallucinations while preserving valid package recommendations. In the package recommendation task, BOUND reduces package-level hallucination rate (Package-HR) by 79.9% on edit prompts and by 65.4% on unseen prompts. The learned package-validity boundary further generalizes to other package-related tasks, reducing Package-HR by 12.8% in code generation and by 34.0% in `pip install` recommendation. These results show that BOUND refines the package-validity boundary of LLMs and improves the reliability of package-related outputs.

I. INTRODUCTION

In recent years, large language models (LLMs) have developed rapidly and demonstrated strong capabilities across various software engineering tasks [1], [2], [3], [4]. Developers increasingly rely on LLMs not only to generate code, but also to recommend libraries, configure dependencies, and prepare development environments [5], [6]. In these workflows, package names generated by LLMs are often treated as actionable recommendations: they may be copied into import statements, installed through package managers, or used by automated coding agents. Consequently, the reliability of package-related outputs has become an important requirement for the practical adoption of LLMs in software development [7], [8].

However, recent studies have shown that LLMs may suffer from hallucinations, generating outputs that appear plausible but are factually incorrect [9], [10], [11]. One particularly concerning type is *package hallucination*, where models generate non-existent or invalid software package names [12], [13], [14]. Unlike many conventional code generation errors, package hallucinations can directly affect dependency management and software supply chain security [15], [16],

[17]. Developers and automated agents may inadvertently install hallucinated packages generated by LLMs, creating opportunities for attackers to register malicious packages under the same names [12]. Such risks are closely related to package confusion attacks, a well-known threat to software supply chains [18], [19]. As LLM-based coding assistants and autonomous agents are increasingly integrated into development workflows, mitigating package hallucination is essential for building trustworthy software engineering environments.

Existing studies have proposed various strategies to mitigate package hallucinations, ranging from external mitigation techniques to internal model adaptation approaches [12]. External mitigation methods primarily operate outside the model. One line of work augments generation with external knowledge sources (e.g., PyPI [20]) to ground model outputs in reliable package information [21], [22]. Another line of work encourages the model to refine its own responses through self-correction [23], [24]. Although these methods can reduce hallucinations, they do not directly modify the model’s internal behavior and introduce additional inference overhead or pipeline complexity [25]. Internal adaptation methods (e.g., supervised fine-tuning) provide a more direct alternative by modifying the model itself [12], [26]. However, fine-tuning updates a large number of parameters and may overfit to specific output formats, making it computationally expensive and potentially degrading the model’s general capabilities. These limitations motivate the need for a lightweight approach that can directly mitigate package hallucinations in LLMs without extensive retraining or architectural modifications.

Recently, model editing, also known as knowledge editing, has been proposed to update specific knowledge or behaviors in LLMs without full retraining [27], [28], [29], [30]. This provides a promising direction for mitigating package hallucination. However, recent studies have shown that existing editing methods may still struggle to correct hallucinated outputs [31]. Existing model editing methods primarily focus on factual knowledge updates, such as modifying subject-object associations or correcting individual facts [32], [33], [34], [35]. These methods often rely on clear subject tokens or target phrases to locate and edit the corresponding knowledge. Package hallucination presents a different challenge. Package-related prompts usually contain complex task descriptions rather than explicit factual subjects, and the desired behavior is not to generate a fixed target output. Instead, the model should suppress non-existent package names while preserving valid packages. Furthermore, valid and hallucinated packages may appear together in the same response. This suggests

that package hallucination is better viewed as a package-validity boundary problem than a factual rewriting problem. We define the package-validity boundary as the model’s ability to distinguish valid packages (i.e., packages that exist in the package ecosystem) from hallucinated packages under a given task context, which is represented by the dashed separation line in Figure 1.

In this paper, we introduce **BOUND**, a lightweight localized model editing framework for mitigating package hallucination in LLMs. Unlike existing mitigation methods that rely on external package retrieval or full-model fine-tuning, BOUND aims to refine the package-validity boundary within the model itself. Specifically, BOUND first employs a risk-aware localization strategy to identify modules whose parameters are most sensitive to the model’s preference for hallucinated over valid packages, which serves as a signal of the package-validity boundary. BOUND then injects lightweight LoRA adapters into the localized modules and optimizes them with a boundary-aware objective that reinforces valid packages, suppresses hallucinated packages, and preserves the original model behavior on locality prompts.

To evaluate the effectiveness of BOUND, we conduct experiments on a package hallucination dataset derived from package-related prompts [12]. We evaluate three representative open-source LLMs: DeepSeekCoder, Qwen3, and Llama-3.1. To provide a comprehensive evaluation, we compare BOUND with five baselines, including two package hallucination mitigation methods (i.e., Full-FT [12] and Self-Refinement [12]) and three model editing methods (i.e., ROME [34], MEMIT [36], and DINM [28]). Experimental results show that ❶ BOUND effectively mitigates package hallucinations in the package recommendation task, reducing package-level hallucination rate (Package-HR) by 79.9% on the edit set, and by 65.4% on unseen prompts. ❷ The learned package-validity boundary generalizes to other package-related tasks, reducing Package-HR by 12.8% in code generation and by 34.0% in `pip install` recommendation. ❸ Compared with existing baselines, BOUND achieves a better balance between hallucination reduction and valid package preservation, while several baselines reduce hallucinations by suppressing package generation. ❹ Ablation results show that both module localization and the three editing objectives are essential to effective editing. ❺ BOUND is lightweight in practice, requiring only 731 KB of LoRA parameters on average, approximately 20,000 $\times$ smaller than full fine-tuning checkpoints.

Contributions: The main contributions of this paper can be summarized as follows:

- We formulate package hallucination as a *package-validity boundary* editing problem, aiming to refine the model’s boundary between valid and hallucinated packages.
- We introduce **BOUND**, a lightweight localized model editing framework that identifies modules related to package hallucination and applies boundary-aware LoRA editing to reduce hallucinated packages while preserving valid package recommendations.

- We conduct experiments on three open-source LLMs and show that BOUND effectively reduces package hallucination, generalizes to other package-related tasks, and requires only lightweight editing artifacts.

II. BACKGROUND

A. Motivating Example

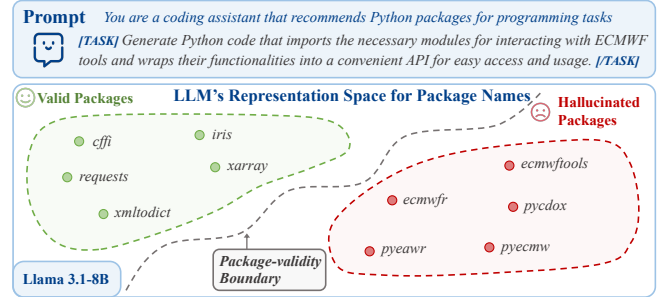


Fig. 1: Motivating example of package hallucination as a package-validity boundary problem.

Figure 1 shows a representative package hallucination example from Llama-3.1-8B. Given a prompt asking for Python packages to interact with ECMWF tools and wrap their functionalities into a convenient API, the model generates both valid packages, such as `cffi` and `iris`, and non-existent package names, such as `ecmwfr` and `pyeawr`. Because these hallucinated names appear plausible under the task context, developers or automated coding agents may mistakenly treat them as valid dependencies, leading to installation failures or software supply chain risks if attackers register malicious packages under the same names.

This example shows that package hallucination is not simply caused by a lack of package knowledge. The model can identify some valid dependencies, but it still fails to distinguish them from plausible non-existent alternatives. We view package hallucination as a package-validity boundary error, where the model’s distinction between valid and hallucinated packages is unreliable under a given task context. Consequently, mitigation should refine this boundary rather than rewrite a specific package recommendation into a fixed target.

B. Task Definition

We formulate the task of package hallucination mitigation through model editing in the context of package recommendation. Let M_θ denote a pretrained LLM with parameters θ . Given a package recommendation prompt x , the model generates an answer $a \sim M_\theta(x)$, from which we extract a set of package names $P(a)$. Based on a package-validity checking procedure, the extracted packages are partitioned into valid packages $G(a)$ and hallucinated packages $H(a)$:

$$P(a) = G(a) \cup H(a), \quad G(a) \cap H(a) = \emptyset. \quad (1)$$

A package hallucination occurs when $H(a) \neq \emptyset$, meaning that the answer contains at least one hallucinated package. For

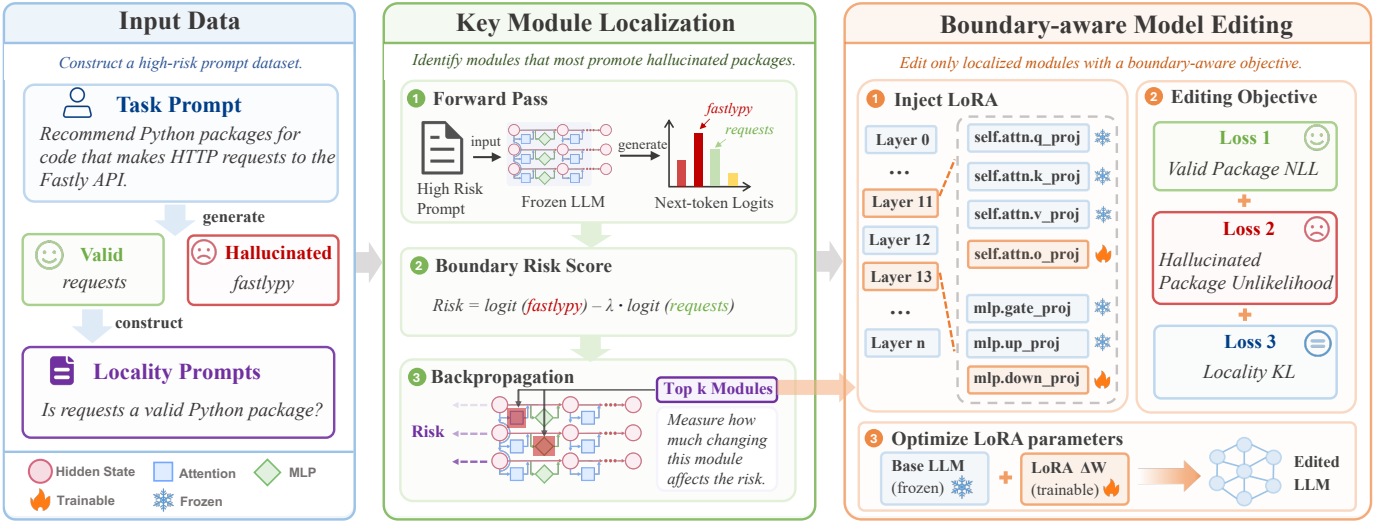


Fig. 2: Overview of BOUND. ❶ (left) BOUND constructs edit cases by identifying valid packages, hallucinated packages, and building locality prompts from high-risk task prompts. ❷ (middle) BOUND computes a boundary risk score and backpropagates it through the frozen LLM to locate modules that most affect the package-validity boundary. ❸ (right) BOUND injects LoRA into the localized modules and optimizes them with complementary objectives to reinforce valid packages, suppress hallucinated packages, and preserve locality behavior.

the i -th case for model editing, let $a_i \sim M_\theta(x_i)$ be the original model response. We define the edit case as

$$e_i = (x_i, G_i, H_i), \quad G_i = G(a_i), \quad H_i = H(a_i). \quad (2)$$

where G_i denotes valid package recommendations to be preserved or encouraged, and H_i denotes hallucinated package recommendations to suppress.

The goal is to learn an edit Δ such that the edited model $M_{\theta+\Delta}$ reduces the likelihood of generating hallucinated packages while preserving valid package recommendations:

$$M_{\theta+\Delta}(x_i) \rightarrow G_i, \quad M_{\theta+\Delta}(x_i) \not\rightarrow H_i. \quad (3)$$

III. APPROACH

A. Overview

Given a set of high-risk package recommendation prompts, BOUND aims to edit the model to reduce hallucinated package names while preserving valid package recommendations. We formulate this goal as refining the model’s package-validity boundary, i.e., its ability to distinguish valid packages from hallucinated package names under a given task context. As shown in Figure 2, BOUND consists of two editing components. First, key module localization identifies modules that consistently contribute to hallucinated package generation across multiple edit cases. Second, model editing injects lightweight LoRA parameters into the localized modules and optimizes them with a boundary-aware objective that reinforces valid packages, suppresses hallucinated packages, and preserves locality behavior. The following subsections describe these two components in detail.

B. Key Module Localization

The objective of this stage is to localize where model edits should be applied. Rather than updating all model parameters,

BOUND identifies modules that are most sensitive to a package-validity boundary risk score.

An autoregressive transformer-based LLM M_θ typically consists of an embedding layer followed by a stack of L decoder layers $\{L_1, L_2, \dots, L_L\}$. Each decoder layer contains a self-attention block and an MLP block. Prior knowledge editing methods primarily focus on MLP modules, based on evidence that feed-forward modules play an important role in storing and updating factual associations [37], [32], [34], [36]. However, package hallucination is not merely a factual knowledge storage problem. Package recommendation requires the model to understand the task context, identify useful dependencies, and distinguish valid package names from non-existent names. Therefore, package-validity boundary errors may involve not only MLP modules that affect vocabulary-level prediction, but also attention modules that route task-relevant contextual information.

To explicitly define the localization space, consider the ℓ -th decoder layer. Given the hidden state h_ℓ , the layer can be written as:

$$\begin{aligned} \tilde{h}_\ell &= h_\ell + \underbrace{W_\ell^o \text{Attn}(W_\ell^q h_\ell, W_\ell^k h_\ell, W_\ell^v h_\ell)}_{\text{Attention block}}, \\ h_{\ell+1} &= \tilde{h}_\ell + \underbrace{W_\ell^d \left(\sigma(W_\ell^g \tilde{h}_\ell) \odot W_\ell^u \tilde{h}_\ell \right)}_{\text{MLP block}}. \end{aligned} \quad (4)$$

For clarity, we omit layer normalization and other implementation-specific details. Here, W_ℓ^q , W_ℓ^k , W_ℓ^v , and W_ℓ^o denote the query, key, value, and output projections in the attention block, respectively. Similarly, W_ℓ^g , W_ℓ^u , and W_ℓ^d denote the gate, up, and down projections in the MLP block, respectively. BOUND treats these projection matrices as candidate modules:

$$\mathcal{M}_\ell = \{W_\ell^q, W_\ell^k, W_\ell^v, W_\ell^o, W_\ell^g, W_\ell^u, W_\ell^d\}. \quad (5)$$

Across all decoder layers, the complete candidate module space is defined as

$$\mathcal{M} = \bigcup_{\ell=1}^L \mathcal{M}_\ell. \quad (6)$$

During localization, the base model remains frozen. The weights of candidate modules are temporarily marked as gradient-bearing only for sensitivity measurement, and no parameter update is performed in this stage. Given a set of high-risk package recommendation prompts ($\mathcal{E}_{\text{loc}}$) for localization, BOUND follows three steps to identify modules that are most sensitive to package-validity boundary risk:

❶ **Forward Pass.** For each edit case $e_i = (x_i, G_i, H_i)$ in $\mathcal{E}_{\text{loc}}$, BOUND feeds the package recommendation prompt x_i into the frozen model M_θ and performs a forward pass to assess the model’s preference among package name candidates at the beginning of the answer. Specifically, BOUND extracts the next-token logits:

$$z_i = M_\theta(x_i)_{\text{next}}, \quad (7)$$

where $z_i \in \mathbb{R}^{|\mathcal{V}|}$ denotes the logit vector over the model vocabulary $\mathcal{V}$. Since the model predicts over tokens rather than package name strings, BOUND maps G_i and H_i to token-level candidate sets G_i^{tok} and H_i^{tok} using the tokenizer, and extracts their logits from z_i as $z_i[G_i^{\text{tok}}]$ and $z_i[H_i^{\text{tok}}]$.

❷ **Boundary Risk Score.** BOUND then constructs a boundary risk score by contrasting hallucinated package candidates with valid package candidates:

$$R_i = \text{LSE}(z_i[H_i^{\text{tok}}]) - \lambda_a \text{LSE}(z_i[G_i^{\text{tok}}]), \quad (8)$$

where $\text{LSE}(\cdot)$ computes a smooth aggregate score over a set of token logits, and λ_a controls the weight of the valid package anchor term. The first term quantifies the model’s preference for hallucinated package names, whereas the second term serves as a valid package anchor that offsets this preference. Consequently, a larger value of R_i indicates a stronger model preference for hallucinated packages relative to valid packages under the current task context.

❸ **Backpropagation.** After computing the boundary risk score, BOUND backpropagates R_i through the frozen model to estimate the sensitivity of each candidate module to the package-validity boundary. For a candidate module $m \in \mathcal{M}$ with weight matrix W_m , BOUND computes a saliency score:

$$s_i(m) = \text{Mean}(|\nabla_{W_m} R_i \odot W_m|), \quad (9)$$

where $\nabla_{W_m} R_i$ denotes the gradient of R_i with respect to W_m , and $\odot$ denotes element-wise multiplication. This score combines the gradient signal with the current weight values and summarizes their absolute effect at the module level. A higher score indicates that module m has a stronger influence on the package-validity boundary and is therefore a more promising editing target. BOUND then averages the saliency scores over all localization cases:

$$S(m) = \frac{1}{|\mathcal{E}_{\text{loc}}|} \sum_{e_i \in \mathcal{E}_{\text{loc}}} s_i(m), \quad (10)$$

BOUND ranks candidate modules by $S(m)$ and selects the top modules as the localized module set $\mathcal{S}$ for subsequent editing.

C. Boundary-aware Model Editing

After localizing the key modules, BOUND updates only the localized module set $\mathcal{S}$ instead of the entire model. This stage refines the model’s package-validity boundary by suppressing hallucinated package names while preserving valid package recommendations. BOUND performs the editing process in three steps:

❶ **Injecting LoRA.** Given the localized module set $\mathcal{S}$, BOUND injects lightweight LoRA adapters into each selected linear module [38]. For a module $m \in \mathcal{S}$, let h_m denote the input hidden representation and o_m denote the module output. The original module computes

$$o_m = W_m h_m, \quad (11)$$

where W_m is the weight matrix. After LoRA injection, the module output is modified as follows:

$$o'_m = W_m h_m + \frac{\alpha}{r} B_m A_m \text{Dropout}(h_m), \quad (12)$$

where A_m and B_m are trainable low-rank matrices, r denotes the LoRA rank, and α is a scaling factor. During editing, the original weight matrix W_m remains frozen, and only the injected LoRA parameters are optimized. The LoRA branch is initialized to produce zero output, so the edited model initially preserves the behavior of the original model. We denote the collection of all LoRA updates on localized modules as Δ .

❷ **Editing Objective.** For each edit case $e_i = (x_i, G_i, H_i)$, BOUND jointly optimizes three complementary objectives:

- **Valid Package NLL.** BOUND uses a negative log-likelihood (NLL) loss on the formatted valid package answer y_i as positive supervision. We construct the target y_i by formatting the valid packages G_i , but it is not assumed to include all correct packages for the task. Instead, it encourages the edited model to keep valid package names under the prompt x_i :

$$\mathcal{L}_{\text{valid}}^{(i)} = -\frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \log P_{\theta+\Delta}(y_{i,t} \mid x_i, y_{i,<t}), \quad (13)$$

where $y_{i,t}$ is the t -th token of y_i , $y_{i,<t}$ is its preceding target context, and $P_{\theta+\Delta}$ denotes the edited model. Minimizing this loss preserves valid package generation and prevents the model from reducing hallucinations by simply suppressing package outputs.

- **Hallucinated Package Unlikelihood.** BOUND uses an unlikelihood loss [39] to explicitly suppress hallucinated package names in H_i . For each hallucinated package $h \in H_i$, BOUND treats it as a candidate continuation after the answer prefix π , where π is the prefix used to start the package list. Let $p_{\theta+\Delta}(h \mid x_i, \pi)$ denote the normalized sequence probability of generating h under

the edited model. BOUND penalizes hallucinated packages that remain after editing:

$$\mathcal{L}_{\text{hall}}^{(i)} = -\frac{1}{|H_i|} \sum_{h \in H_i} \log(1 - p_{\theta+\Delta}(h \mid x_i, \pi)), \quad (14)$$

minimizing this objective pushes the model to reduce the probability of hallucinated package names.

- **Locality KL.** BOUND uses a locality loss based on Kullback-Leibler (KL) divergence to reduce unintended changes caused by editing. For each edit case, we construct a locality set C_i from two sources. First, BOUND derives package-related locality prompts from the observed valid packages in G_i to preserve the model’s behavior around valid package knowledge. Second, BOUND includes the original task prompt x_i to prevent the edited model from overfitting to the formatted target answer y_i . For each locality prompt $c \in C_i$, BOUND encourages the edited model to preserve the next token distribution of the base model:

$$\mathcal{L}_{\text{loc}}^{(i)} = \frac{1}{|C_i|} \sum_{c \in C_i} D_{\text{KL}}(P_{\theta}(\cdot \mid c) \parallel P_{\theta+\Delta}(\cdot \mid c)), \quad (15)$$

Here, $P_{\theta}(\cdot \mid c)$ and $P_{\theta+\Delta}(\cdot \mid c)$ denote the next token distributions of the original and edited models under prompt c . Minimizing this loss constrains the edited model and reduces distribution drift on valid package contexts and the original task context.

Overall, the editing objective is

$$\mathcal{L}_{\text{BOUND}} = \frac{1}{|\mathcal{E}_{\text{edit}}|} \sum_{e_i \in \mathcal{E}_{\text{edit}}} \left(\lambda_v \mathcal{L}_{\text{valid}}^{(i)} + \lambda_h \mathcal{L}_{\text{hall}}^{(i)} + \lambda_l \mathcal{L}_{\text{loc}}^{(i)} \right), \quad (16)$$

where λ_v , λ_h , and λ_l control the strengths of valid package reinforcement, hallucinated package suppression, and locality preservation, respectively.

③ **Optimizing LoRA parameters.** BOUND minimizes $\mathcal{L}_{\text{BOUND}}$ by updating the LoRA parameters injected into the localized module set $\mathcal{S}$, while keeping the base parameters θ fixed. After optimization, the learned LoRA updates form the edit Δ , and the edited model is denoted as $M_{\theta+\Delta}$. By combining localized updates with valid package reinforcement, hallucinated package suppression, and locality preservation, BOUND refines the model’s package-validity boundary instead of merely fitting valid package targets.

IV. EXPERIMENTAL SETUP

A. Dataset

We construct a high-risk prompt set for each evaluated model and further split it into edit and unseen evaluation subsets. The dataset construction process contains three steps:

① **Prompt source.** We follow the LLM-generated prompt dataset released by Spracklen et al. [12] and use its recent Python subset. The prompts are derived from descriptions of popular Python packages and cover diverse package-related programming tasks.

TABLE I: Statistics of high-risk prompt sets for the evaluated models.

Model	High-risk Prompt	Edit Prompt	Evaluation Prompt
DeepSeekCoder	741	4×50	541
Qwen3	609	4×50	409
Llama-3.1	1,134	4×50	934

② **High-risk prompt construction.** Since package hallucination behavior varies across models, we construct a separate high-risk prompt set for each evaluated model. For each prompt x , we query the model five times in the package recommendation setting, extract the recommended packages, and check their validity using PyPI metadata [20]. A package is valid only if it exists on PyPI and has at least one release before the model’s knowledge cutoff. Otherwise, it is classified as a hallucinated package. We then compute the sample-level hallucination rate (Sample-HR) of each prompt over the five generations, as defined in Equation 17. We retain a prompt as a high-risk prompt if Sample-HR > 0.5 , focusing our study on prompts that consistently trigger package hallucinations.

③ **Dataset split.** Table I summarizes the resulting high-risk prompt sets. For each model, we split high-risk prompts into four edit subsets and one unseen evaluation subset. Each edit subset contains 50 prompts and is used independently for editing, reducing the effect of randomness in edit case selection. For each edit prompt x_i , we construct an edit case $e_i = (x_i, G_i, H_i)$, where G_i and H_i are the extracted valid and hallucinated packages from the five generations. The remaining high-risk prompts are used to evaluate whether the package-validity boundary learned through editing can generalize to unseen prompts.

B. Baselines

To evaluate the effectiveness of BOUND, we compare it with five baselines, including two package hallucination mitigation methods and three model editing methods. For a fair comparison, all trainable baselines use the same high-risk edit cases as BOUND.

Full-FT [12] is a direct model adaptation baseline for mitigating package hallucinations. We construct supervised examples by pairing each package recommendation prompt x_i with its valid package answer y_i , and fine-tune the model to increase the likelihood of valid package recommendations.

Self-Refinement [12] is an inference-time baseline that iteratively verifies and refines package recommendations. In each round, the model generates packages, checks their validity, adds invalid packages to an avoidance list, and regenerates the answer. We repeat this process for up to five rounds.

ROME [34] is a localized knowledge editing method that applies rank-one parameter updates to edit factual associations. We adapt ROME by using the package recommendation prompt x_i as the editing context and the valid package answer y_i as the target output.

MEMIT [36] extends ROME to edit multiple factual associations in a batch. We use the same edit cases to evaluate whether batch factual editing can mitigate package hallucination at a larger edit scale.

DINM [28] is a behavior editing method originally designed for tasks such as detoxification. We adapt DINM by treating y_i as the desired behavior, and evaluate whether behavior editing can suppress hallucinated package recommendations while preserving valid packages.

C. Evaluation Metrics

We evaluate each method from two perspectives: package hallucination and valid package preservation. For each generated answer a , we extract the set of recommended packages $P(a)$ and classify each package using the cutoff-aware PyPI validation procedure described in Section IV-A. Let $G(a)$ and $H(a)$ denote the sets of valid and hallucinated packages extracted from a , respectively.

Sample-HR: Sample-level hallucination rate measures the fraction of generated answers that contain at least one hallucinated package:

$$\text{Sample-HR} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(|H(a_i)| > 0), \quad (17)$$

where N is the number of generated answers. Sample-HR reflects how frequently users are exposed to hallucinated package recommendations.

Package-HR: Package-level hallucination rate measures the fraction of hallucinated packages among extracted packages:

$$\text{Package-HR} = \frac{\sum_{i=1}^N |H(a_i)|}{\sum_{i=1}^N (|G(a_i)| + |H(a_i)|)}. \quad (18)$$

This metric reflects the overall hallucination level among generated package names. If no package is extracted from any generated answer, we define Package-HR as 0.

Valid-Rate: Valid-Rate measures the fraction of generated answers that contain at least one valid package:

$$\text{Valid-Rate} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(|G(a_i)| > 0). \quad (19)$$

Here, a valid package means a real package that exists in the package ecosystem. It does not imply that the package is semantically correct for the given task. Valid-Rate complements the hallucination metrics, since a method may reduce hallucinations by suppressing package generation.

D. Experimental Setting

Models. We evaluate on three open-source LLMs: `deepseek-coder-6.7b-instruct` [40], `Qwen3-8B` [41], and `Llama-3.1-8B-Instruct` [42]. All experiments use PyTorch and HuggingFace Transformers, with model weights loaded in `bfloat16` precision.

Editing Hyperparameters. BOUND selects the top 5 modules during localization. During editing, it uses a LoRA rank of $r = 8$, scaling factor $\alpha = 16$, dropout rate of 0.05, batch size of 1, learning rate of 2×10^{-4} , and three training epochs. The loss weights λ_v , λ_h , and λ_l are set to 1.0, 0.15, and 0.03, respectively. For a fair comparison, all trainable baselines

use the same edit cases as BOUND, while method-specific hyperparameters follow their default settings.

Generation Settings. For all evaluations, each prompt is sampled five times using `temperature = 0.7` and `top_p = 0.9`. The maximum generation length is set to 128 tokens for package recommendation and `pip install` recommendation tasks, and 2,048 tokens for code generation.

Hardware. All experiments are conducted on a Linux server running Ubuntu 22.04 LTS, equipped with two Intel Xeon Platinum 8358P CPUs (64 cores total @ 2.60 GHz), 1 TB RAM, and eight NVIDIA A100-SXM4 GPUs (80 GB memory).

V. RESULTS AND ANALYSIS

In this section, we aim to answer the following three research questions:

RQ1 (Effectiveness). How effective is BOUND compared to package hallucination mitigation and model editing methods?

RQ2 (Generalization). Can BOUND generalize from package recommendation to other package-related tasks?

RQ3 (Ablation Study). How does each component of BOUND contribute to its overall effectiveness?

A. RQ1: Effectiveness

To answer RQ1, we evaluate BOUND and all baselines in the package recommendation setting. We also report the unedited model as **Base**. Following the dataset split in Table I, all trainable methods are trained using the same four edit folds, each containing 50 prompts. For each method, we report the average performance across the four folds on both the edit set and the unseen evaluation set. Table II presents the results.

👍 **Effectiveness of BOUND:** BOUND substantially reduces package hallucinations across all three models while preserving valid package recommendations. On the edit prompts, BOUND reduces the average Sample-HR from 0.8500 to 0.2050 (-75.9%) and the average Package-HR from 0.4463 to 0.0897 (-79.9%). Meanwhile, the average Valid-Rate increases from 0.8013 to 0.8527 (+6.4%), indicating that BOUND suppresses hallucinated packages without sacrificing valid package generation. The improvements further generalize to unseen prompts that are not used during editing. On the unseen evaluation prompts, BOUND reduces the average Sample-HR from 0.8573 to 0.3113 (-63.7%) and the average Package-HR from 0.4421 to 0.1529 (-65.4%). These consistent gains suggest that BOUND learns a transferable package-validity boundary rather than simply memorizing the edited prompts.

👍 **Comparison with Baselines:** Compared with model editing baselines, BOUND achieves a better balance between hallucination reduction and valid package preservation. Although ROME and DINM sometimes achieve very low Sample-HR or Package-HR, these gains are often accompanied by substantial drops in Valid-Rate. For example, on Llama-3.1, ROME reduces the Edit50 Sample-HR to 0.1590, but its Valid-Rate drops to only 0.0110. Similar trends can be observed for DINM across multiple models. This suggests that these methods may reduce hallucinations by suppressing package recommendation behavior itself. In contrast, BOUND reduces

TABLE II: Effectiveness comparison in the package recommendation setting. We report results on both Edit50 prompts and unseen evaluation prompts. Lower Sample-HR and Package-HR are better, while higher Valid-Rate is better.

Model	Method	Edit50 Prompts			Unseen Evaluation Prompts		
		Sample-HR ↓	Package-HR ↓	Valid-Rate ↑	Sample-HR ↓	Package-HR ↓	Valid-Rate ↑
DeepSeekCoder	Base	0.8140	0.4091	0.8270	0.8333	0.4414	0.7978
	BOUND	0.1980	0.0745	0.9480	0.3712	0.1706	0.8909
	ROME	0.1820	0.0916	0.2120	0.1879	0.1085	0.1978
	MEMIT	0.7560	0.4049	0.8160	0.7503	0.4114	0.8001
	DINM	0.1540	0.1327	0.4000	0.2805	0.2366	0.3707
	Full-FT	0.0580	0.0188	0.9500	0.4814	0.2141	0.8991
	Self-Refinement	0.5180	0.2263	0.8360	–	–	–
Llama-3.1	Base	0.8320	0.4308	0.8150	0.8368	0.4295	0.8182
	BOUND	0.3180	0.1419	0.8690	0.4033	0.1955	0.8353
	ROME	0.1590	0.4297	0.0110	0.1587	0.4325	0.0147
	MEMIT	0.7340	0.3899	0.8310	0.7389	0.3870	0.8304
	DINM	0.0320	0.0423	0.2710	0.0387	0.0590	0.2591
	Full-FT	0.5110	0.3064	0.8660	0.5592	0.3546	0.8265
	Self-Refinement	0.7120	0.3897	0.7740	–	–	–
Qwen3	Base	0.9040	0.4991	0.7620	0.9017	0.4555	0.7990
	BOUND	0.0990	0.0526	0.7410	0.1595	0.0927	0.7434
	ROME	0.3760	0.1879	0.6640	0.6069	0.4036	0.4475
	MEMIT	0.8440	0.4757	0.7520	0.8318	0.4423	0.7883
	DINM	0.7670	0.4372	0.7480	0.7545	0.4124	0.7791
	Full-FT	0.1460	0.0895	0.7780	0.3341	0.2572	0.6417
	Self-Refinement	0.5650	0.2668	0.6860	–	–	–

hallucinations while maintaining a high Valid-Rate, indicating that it better separates valid packages from hallucinated ones. BOUND also generalizes better than package hallucination mitigation baselines. Although Full-FT performs competitively on some edit sets, its improvements transfer less consistently to unseen prompts. On the unseen evaluation prompts, BOUND reduces Package-HR by 65.4% on average across the three models, compared with 37.8% for Full-FT. Self-Refinement achieves moderate hallucination reductions, but its gains are smaller than those of BOUND (29.6% vs. 75.9% Sample-HR reduction on Edit50). Furthermore, Self-Refinement can be applied to any prompt at inference time. However, since our unseen evaluation focuses on the generalization of the edited model, Self-Refinement is not included in this comparison.



Answer to RQ1: BOUND achieves the best overall balance between hallucinated package mitigation and valid package preservation. On edit prompts, it reduces Sample-HR and Package-HR by 75.9% and 79.9%, while improving Valid-Rate by 6.4%. These improvements also generalize to unseen prompts, reducing Sample-HR and Package-HR by 63.7% and 65.4%.

B. RQ2: Generalization

RQ1 shows the effectiveness of BOUND in the package recommendation task. We further investigate whether the package-validity boundary knowledge learned from package recommendation can transfer to other package-related tasks. We evaluate two representative tasks: code generation and `pip install` recommendation. For code generation, the edited model first generates Python code for a given task. We then extract the `import` modules and use the corresponding

unedited base model to map them to PyPI package names. This design separates package name mapping from code generation quality, allowing the evaluation to focus on how editing affects dependencies in generated code. For `pip install` recommendation, we give the model the task prompt and ask it to generate `pip install` commands directly. We then extract package names from the generated commands for validity checking. To control evaluation cost, we randomly sample 100 prompts from the unseen evaluation prompt set. Following the RQ1 setup, we evaluate the four edited models obtained from the four edit folds and report their average results. We exclude Self-Refinement because it is an inference-time mitigation method and does not produce edited model parameters that can transfer across tasks. Table III presents the results.

✂ Cross-Task Generalization: The edited package-validity boundary transfers from package recommendation to both code generation and `pip install` recommendation. In code generation, BOUND reduces the average Sample-HR and Package-HR by 7.3% and 12.8%, respectively, while improving the average Valid-Rate by 2.8%. This indicates that editing package recommendation behavior can also reduce hallucinated dependencies in generated code. However, the gains are smaller than those in the package recommendation setting. One possible reason is that package hallucinations in code generation are often introduced through hallucinated module imports, making the effect of package-validity boundary editing less direct than in explicit package recommendation. BOUND shows stronger transfer in the `pip install` recommendation setting. Across the three models, it reduces Sample-HR and Package-HR by 35.7% and 34.0%, while

TABLE III: Cross-task generalization results on code generation and `pip install` recommendation.

Model	Method	Code Generation			pip install Recommendation		
		Sample-HR ↓	Package-HR ↓	Valid-Rate ↑	Sample-HR ↓	Package-HR ↓	Valid-Rate ↑
DeepSeekCoder	Base	0.3420	0.2643	0.6420	0.4340	0.2986	0.6320
	BOUND	0.3260	0.2542	0.6415	0.3375	0.1987	0.7690
	DINM	0.3105	0.2714	0.5655	0.2070	0.2352	0.3985
	MEMIT	0.3550	0.2758	0.6215	0.4150	0.3017	0.6130
	ROME	0.0850	0.0671	0.1510	0.1010	0.0669	0.1515
	Full-FT	0.3540	0.2729	0.6385	0.3845	0.2547	0.6275
Qwen3	Base	0.3700	0.2785	0.5500	0.4820	0.3569	0.6040
	BOUND	0.3385	0.2237	0.5815	0.2400	0.2122	0.6030
	DINM	0.3230	0.2682	0.4820	0.5015	0.3704	0.5945
	MEMIT	0.3640	0.2829	0.5200	0.4910	0.3608	0.6050
	ROME	0.0935	0.0715	0.1325	0.1215	0.0871	0.1545
	Full-FT	0.0930	0.1094	0.3200	0.0000	0.0000	0.0005
Llama-3.1	Base	0.3380	0.2425	0.6940	0.5760	0.3452	0.6600
	BOUND	0.3085	0.2070	0.7165	0.3825	0.2492	0.7675
	DINM	0.0095	0.1352	0.0115	0.0005	0.2500	0.0000
	MEMIT	0.3355	0.2319	0.6890	0.5650	0.3348	0.6785
	ROME	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
	Full-FT	0.0000	0.0000	0.0005	0.0000	0.0000	0.0000

improving Valid-Rate by 12.8%. This suggests that the edited package-validity boundary generalizes well to explicit package installation tasks, where the model still needs to determine whether a package name is valid.

✎ **Comparison with Baselines:** ROME, DINM, and Full-FT sometimes achieve very low Sample-HR or Package-HR, but often at the cost of valid package generation. For example, on Llama-3.1 in the `pip install` setting, ROME and Full-FT both achieve zero Sample-HR and Package-HR, while their Valid-Rate also drops to zero. This suggests that these methods may damage the model’s basic ability to perform package-related tasks, leading to fewer package outputs rather than reducing hallucinations. In contrast, BOUND consistently reduces hallucinations while maintaining a high Valid-Rate, indicating that it better preserves valid package behavior. MEMIT preserves Valid-Rate more effectively than ROME and DINM, but its hallucination reduction is limited. In both code generation and `pip install` recommendation, its Sample-HR and Package-HR remain close to those of the Base model. This suggests that MEMIT preserves the original model behavior, but has limited ability to transfer the edited package-validity boundary knowledge across tasks. Overall, these results indicate that directly applying existing methods is insufficient for cross-task package hallucination mitigation, whereas BOUND provides a more reliable package-validity boundary editing approach.



Answer to RQ2: BOUND transfers the package-validity boundary learned from package recommendation to other package-related tasks. It reduces Package-HR by 12.8% in code generation and by 34.0% in `pip install` recommendation. These results suggest that BOUND learns a transferable distinction between valid and hallucinated packages, rather than merely fitting the edited recommendation prompts.

C. RQ3: Ablation Study

To investigate the contribution of each component to the effectiveness of BOUND, we create the following variants:

- **BOUND_Full:** The complete version of BOUND, as described in Section III.
- **w/o Localization:** This variant removes module localization and randomly selects the same number of editable modules as BOUND. It evaluates whether locating modules related to the package-validity boundary is necessary.
- **w/o Valid-NLL:** This variant removes the valid package NLL loss, which uses valid package answers as positive supervision. It evaluates whether valid package learning is needed to preserve useful recommendations.
- **w/o Hall-UL:** This variant removes the hallucinated package unlikelihood loss and no longer directly suppresses hallucinated packages. It evaluates whether explicit hallucination suppression is necessary.
- **w/o Locality-KL:** This variant removes the locality KL loss, so the edited model is no longer constrained to preserve the original behavior on locality prompts. It evaluates whether locality preservation is needed for stable editing.

Following the setup of RQ2, we randomly sample 100 prompts from the unseen prompt set to control evaluation cost. For each variant, we edit the model separately on the four edit sets and report the average results across the four edited models. Table IV presents the results.

Overall, the complete version of BOUND achieves the best overall effectiveness among all variants, balancing hallucination reduction and valid package preservation. Across the three models, it obtains an average Sample-HR of 0.3207, Package-HR of 0.1508, and Valid-Rate of 0.8248. Removing localization increases Sample-HR and Package-HR by 18.3% and 42.8%, while reducing Valid-Rate by 11.5%. This result suggests that editing localized modules is more effective than

TABLE IV: Ablation study of BOUND across three models.

Setting	DeepSeekCoder			Qwen3			Llama-3.1		
	Sample-HR ↓	Package-HR ↓	Valid-Rate ↑	Sample-HR ↓	Package-HR ↓	Valid-Rate ↑	Sample-HR ↓	Package-HR ↓	Valid-Rate ↑
BOUND	0.3620	0.1618	0.8835	0.1300	0.0702	0.7265	0.4700	0.2204	0.8645
w/o Localization	0.4110	0.2356	0.7590	0.2055	0.1548	0.5890	0.5215	0.2555	0.8420
w/o Valid-NLL	0.0200	0.0102	0.2140	0.0000	0.0000	0.0000	0.0855	0.0893	0.2395
w/o Hall-UL	0.6435	0.4728	0.6435	0.5545	0.5706	0.3205	0.7575	0.6505	0.3980
w/o Locality-KL	0.6420	0.5668	0.4625	0.5065	0.5214	0.3175	0.7120	0.6027	0.4190

TABLE V: Average cost of BOUND and baselines across models.

Method	Loc. time (s)	Edit time (s)	Test time (s)	Artifact size
BOUND	23.81	83.09	11.30	731.03 KB
ROME	10.19	384.45	10.86	98.01 MB
MEMIT	—	193.06	13.17	588.01 MB
DINM	0.92	11.97	12.63	126.68 MB
Full-FT	—	124.16	15.31	14.27 GB
Self-Refinement	—	—	25.12	—

editing randomly selected modules. The three editing objectives also contribute to this balance. Removing Valid-NLL reduces Valid-Rate from 0.8248 to 0.1512 (-81.7%), indicating that the model tends to suppress package generation instead of preserving valid packages. Removing Hall-UL increases Package-HR from 0.1508 to 0.5646 (+274.4%), showing that suppressing hallucinated package names is necessary for hallucination reduction. Removing Locality-KL increases Package-HR to 0.5636 (+273.8%) and reduces Valid-Rate to 0.3997 (-51.5%), suggesting that locality preservation helps stabilize editing and prevents harmful behavioral drift.



Answer to RQ3: Each component contributes to the effectiveness of BOUND. Module localization identifies modules sensitive to the package-validity boundary, while the three loss terms work together to balance hallucination suppression and valid package preservation.

VI. DISCUSSION

A. Cost of BOUND

To assess the practical cost of BOUND, we evaluate its runtime and storage overhead. Table V reports the average results across the three evaluated models. BOUND introduces a localization step that takes 23.81 seconds on average. Although this adds extra cost, RQ3 shows that localization improves editing effectiveness by identifying modules related to the package-validity boundary. For editing, BOUND requires 83.09 seconds on average, which is lower than ROME, MEMIT, and Full-FT. At inference time, BOUND takes 11.30 seconds per case on average, which is comparable to ROME and lower than other baselines. BOUND is also lightweight in storage, requiring only 731.03 KB of artifacts on average, approximately 20,000× smaller than full fine-tuning checkpoints. This is because BOUND stores only LoRA updates on localized modules rather than large parameter edits or full checkpoints. Overall, BOUND adds a small localization cost while achieving efficient editing, competitive inference time, and very small storage overhead. These results show that BOUND is practical for mitigating package hallucination.

TABLE VI: Code accuracy on HumanEval before and after editing.

Model	pass@1			pass@10		
	Orig.	Edited	Δ	Orig.	Edited	Δ
DeepSeekCoder	68.96%	67.58%	-1.38%	82.32%	83.39%	+1.07%
Qwen3	23.29%	21.93%	-1.36%	51.83%	50.91%	-0.92%
Llama-3.1	41.52%	42.12%	+0.60%	69.51%	69.66%	+0.15%

B. Impact of Editing on Model Performance

While BOUND effectively reduces package hallucination, it is important to examine whether editing affects the model’s general code generation ability. To this end, we evaluate the original and edited models on HumanEval. Following the main evaluation setup, each model has four edited versions, obtained by editing on the four edit folds separately. We evaluate all four edited models on HumanEval and report the average pass@1 and pass@10 results. Table VI presents the results.

The results show that BOUND remains stable on clean code generation inputs, with most changes within a $\pm 2\%$ margin. Overall, BOUND mitigates package hallucination with minimal impact on the original capabilities of the model.

C. Threats to Validity

Internal Validity: A potential internal threat comes from package extraction. We rely on regular expressions to extract package names from model outputs. Due to the inherent instability of LLM outputs, the extracted packages may not always follow the expected format. To reduce this threat, we constrain the output format in the prompt and use one-shot prompting to guide generation. Another concern is the stability of evaluation. Since LLM generation involves randomness, the measured hallucination rates may exhibit slight variance. We address this by generating five responses for each prompt, and reporting average results over four edit folds. The module localization step of BOUND may also introduce uncertainty. Although BOUND identifies modules related to package boundaries using a risk-aware localization objective, the results may still be affected by noisy gradients or the selected edit cases. To mitigate this threat, we aggregate localization signals over multiple edit cases.

External Validity: Our evaluation is conducted on three open-source LLMs from different model families. However, the results may not directly generalize to other LLMs. We focus on open-source models because they provide stable access after release, allowing historical versions to be revisited and reproduced. In contrast, commercial LLMs are continuously updated and may become unavailable over time. Another external threat comes from the package ecosystem studied in this paper. We focus on Python packages from PyPI because PyPI

provides public metadata for package validity checking. Future work can extend BOUND to additional package ecosystems and programming languages (e.g., Java) to further evaluate its generality. Furthermore, we evaluate BOUND on three package-related tasks: package recommendation, code generation, and `pip install` recommendation. Although these tasks cover common package-related development scenarios, real-world dependency management also involves configuration files, build scripts, and repository-level workflows. Future work can extend BOUND to these broader settings.

VII. RELATED WORK

A. Package Hallucination

Prior work has studied hallucination in natural language generation and general LLM settings, such as dialogue, summarization, and factual question answering [43], [25], [9], [44]. Recent software engineering studies show that hallucination also appears in code generation, where models may produce incorrect APIs, non-existent libraries, or flawed implementation logic [45], [46], [11]; documentation grounding has also been explored to mitigate API hallucinations [22]. Package hallucination is a specific failure mode where an LLM recommends dependency names that do not exist or are unavailable in the target ecosystem. Empirical studies show that package hallucination is common across models, programming languages, model sizes, and task settings [12], [13], and HFuzzer further triggers such failures through phrase-based fuzzing [14]. This problem also raises software supply chain risks, since developers or coding agents may pass fabricated names to package managers, creating opportunities for package confusion, slop squatting, or malicious registration [16], [17], [19], [18], [12], [13].

Researchers have proposed several strategies to mitigate hallucinations [26], [44]. Retrieval methods ground generation with external knowledge sources, self-correction prompts models to verify and refine their outputs, and model adaptation changes model behavior through fine-tuning or similar training procedures [22], [23], [24], [12]. Inference-time and post-hoc editing methods, such as TruthX and PURR, mitigate hallucinations by intervening in internal representations or revising generated outputs without full retraining [47], [48]. However, these methods often depend on external resources, require repeated inference, revise outputs without correcting the model’s default behavior, or incur high training costs. In contrast, we address package hallucination through localized model editing with a small set of editing examples.

B. Model Editing

Model editing, also known as knowledge editing, aims to correct specific knowledge or behaviors in pretrained models without full retraining [49], [50], [28]. A successful edit should fix the target error while preserving the model’s original capabilities, and is commonly evaluated by reliability, generalization, locality, and portability [29], [51]. Existing methods can be broadly divided into two groups. Weight-preserving methods, such as SERAC, GRACE, and IKE, rely

on external memories or in-context demonstrations to override selected predictions without modifying model parameters [52], [53], [54]. Weight-modified methods directly update model parameters through learned editors or localized changes. For example, KnowledgeEditor and MEND learn auxiliary networks to generate parameter updates [55], [56], while later studies show that aggressive or sequential editing may harm general abilities or cause forgetting [57], [58].

Among weight-modified methods, localized editing methods are especially relevant to our work. Motivated by evidence that knowledge is encoded in specific transformer components [37], [32], ROME, MEMIT, and PMET edit selected internal modules to rewrite factual associations at different scales [34], [36], [33]. Recent work further extends model editing beyond factual triples, including detoxification, code LLM robustness, and editing for LLMs4Code [28], [30], [27]. However, package hallucination remains unexplored in model editing. Unlike factual updates that rewrite isolated subject-object associations, package hallucination requires correcting a task-conditioned package-validity boundary over package names. Our work targets this package-validity boundary editing problem.

VIII. CONCLUSION

In this paper, we presented **BOUND**, a lightweight localized model editing framework for mitigating package hallucination in LLMs. BOUND formulates package hallucination as a package-validity boundary editing problem, where the goal is to improve the model’s distinction between valid and hallucinated package names. It first locates modules related to package hallucination through risk-aware localization, and then edits only these modules with lightweight LoRA adapters and a boundary-aware objective. We evaluated BOUND on three representative open-source LLMs: DeepSeekCoder, Qwen3, and Llama-3.1. The results show that BOUND effectively reduces package hallucination in package recommendation, lowering Package-HR by 79.9% on edit prompts and by 65.4% on unseen prompts. The edited package-validity boundary also transfers to other package-related tasks, reducing Package-HR by 12.8% in code generation and by 34.0% in `pip install` recommendation. Compared with existing baselines, BOUND better balances hallucination reduction and valid package preservation, while several baselines reduce hallucinations by suppressing package generation. Ablation and cost analyses further show that each component contributes to BOUND’s effectiveness, and the framework is practical and lightweight. Overall, BOUND provides a practical step toward mitigating package hallucinations in LLMs without full model retraining or external retrieval pipelines. Future work will extend BOUND to more package ecosystems and broader dependency management scenarios.

IX. DATA AVAILABILITY STATEMENT

The dataset and code are publicly available in our replication package [59], [60].

REFERENCES

- [1] J. Wang and Y. Chen, "A review on code generation with llms: Application and evaluation," in *2023 IEEE International Conference on Medical Artificial Intelligence (MedAI)*. IEEE, 2023, pp. 284–289.
- [2] C. E. Jimenez, J. Yang, A. Wetteg, S. Yao, K. Pei, O. Press, and K. Narasimhan, "Swe-bench: Can language models resolve real-world github issues?" in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 54 107–54 157.
- [3] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez *et al.*, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.
- [4] W. Sun, Y. Miao, Y. Li, H. Zhang, C. Fang, Y. Liu, G. Deng, Y. Liu, and Z. Chen, "Source code summarization in the era of large language models," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025, pp. 1882–1894.
- [5] M. Alhanahnah and Y. Boshmaf, "Depsrags: Towards agentic reasoning and planning for software dependency management," *arXiv preprint arXiv:2405.20455*, 2024.
- [6] J. Latendresse, S. Khatoonabadi, and E. Shihab, "How robust are llm-generated library imports? an empirical study using stack overflow," *arXiv preprint arXiv:2507.10818*, 2025.
- [7] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of github copilot's code contributions," *Communications of the ACM*, vol. 68, no. 2, pp. 96–105, 2025.
- [8] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do users write more insecure code with ai assistants?" in *Proceedings of the 2023 ACM SIGSAC conference on computer and communications security*, 2023, pp. 2785–2799.
- [9] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen *et al.*, "Siren's song in the ai ocean: A survey on hallucination in large language models," *Computational Linguistics*, vol. 51, no. 4, pp. 1373–1418, 2025.
- [10] C. Gao, X. Hu, S. Gao, X. Xia, and Z. Jin, "The current challenges of software engineering in the era of large language models," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 1–30, 2025.
- [11] Z. Zhang, C. Wang, Y. Wang, E. Shi, Y. Ma, W. Zhong, J. Chen, M. Mao, and Z. Zheng, "Llm hallucinations in practical code generation: Phenomena, mechanism, and mitigation," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 481–503, 2025.
- [12] J. Spracklen, R. Wijewickrama, A. N. Sakib, A. Maiti, and B. Viswanath, "We have a package for you! a comprehensive analysis of package hallucinations by code generating LLMs," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 3687–3706.
- [13] A. Krishna, E. Galinkin, L. Derczynski, and J. Martin, "Importing phantoms: Measuring llm package hallucination vulnerabilities," *arXiv preprint arXiv:2501.19012*, 2025.
- [14] Y. Zhao, M. Wu, X. Hu, and X. Xia, "Hfuzzer: Testing large language models for package hallucinations via phrase-based fuzzing," *arXiv preprint arXiv:2509.23835*, 2025.
- [15] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's knife collection: A review of open source software supply chain attacks," in *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, 2020, pp. 23–43.
- [16] P. Ladisa, H. Plate, M. Martinez, and O. Barais, "Sok: Taxonomy of attacks on open-source software supply chains," in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 1509–1526.
- [17] S. Neupane, G. Holmes, E. Wyss, D. Davidson, and L. De Carli, "Beyond typosquatting: an in-depth look at package confusion," in *32nd USENIX security symposium (USENIX security 23)*, 2023, pp. 3439–3456.
- [18] B. Lanyado, "Diving deeper into ai package hallucinations," <https://www.lasso.security/blog/ai-package-hallucinations>, 2024.
- [19] B. Kaplan and J. Qian, "A survey on common threats in npm and pypi registries," in *International Workshop on Deployable Machine Learning for Security Defense*. Springer, 2021, pp. 132–156.
- [20] Python Software Foundation, "PyPI: The python package index," <https://pypi.org/>, 2026, accessed: 2026-06-14.
- [21] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in neural information processing systems*, vol. 33, pp. 9459–9474, 2020.
- [22] N. Jain, R. Kwiatkowski, B. Ray, M. K. Ramanathan, and V. Kumar, "On mitigating code llm hallucinations with api documentation," in *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 2025, pp. 237–248.
- [23] Z. Ji, T. Yu, Y. Xu, N. Lee, E. Ishii, and P. Fung, "Towards mitigating llm hallucination via self reflection," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 1827–1843.
- [24] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang *et al.*, "Self-refine: Iterative refinement with self-feedback," *Advances in neural information processing systems*, vol. 36, pp. 46 534–46 594, 2023.
- [25] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin *et al.*, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Transactions on Information Systems*, vol. 43, no. 2, pp. 1–55, 2025.
- [26] S. Tonmoy, S. Zaman, V. Jain, A. Rani, V. Rawte, A. Chadha, and A. Das, "A comprehensive survey of hallucination mitigation techniques in large language models," *arXiv preprint arXiv:2401.01313*, 2024.
- [27] X. Li, S. Wang, S. Li, J. Ma, J. Yu, X. Liu, J. Wang, B. Ji, and W. Zhang, "Model editing for llms4code: How far are we?" in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025, pp. 937–949.
- [28] M. Wang, N. Zhang, Z. Xu, Z. Xi, S. Deng, Y. Yao, Q. Zhang, L. Yang, J. Wang, and H. Chen, "Detoxifying large language models via knowledge editing," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 3093–3118.
- [29] N. Zhang, Y. Yao, B. Tian, P. Wang, S. Deng, M. Wang, Z. Xi, S. Mao, J. Zhang, Y. Ni *et al.*, "A comprehensive study of knowledge editing for large language models," *arXiv preprint arXiv:2401.01286*, 2024.
- [30] S. Liu, X. Hu, K. Huang, X. Yang, D. Lo, and X. Xia, "Creme: Robustness enhancement of code llms via layer-aware model editing," *arXiv preprint arXiv:2507.16407*, 2025.
- [31] B. Huang, C. Chen, X. Xu, A. Payani, and K. Shu, "Can knowledge editing really correct hallucinations?" in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 88 116–88 149.
- [32] D. Dai, L. Dong, Y. Hao, Z. Sui, B. Chang, and F. Wei, "Knowledge neurons in pretrained transformers," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2022, pp. 8493–8502.
- [33] X. Li, S. Li, S. Song, J. Yang, J. Ma, and J. Yu, "Pmet: Precise model editing in a transformer," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 18 564–18 572.
- [34] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, "Locating and editing factual associations in gpt," *Advances in neural information processing systems*, vol. 35, pp. 17 359–17 372, 2022.
- [35] X. Wu, J. Li, M. Xu, W. Dong, S. Wu, C. Bian, and D. Xiong, "Depn: Detecting and editing privacy neurons in pretrained language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 2875–2886.
- [36] K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau, "Mass-editing memory in a transformer," *arXiv preprint arXiv:2210.07229*, 2022.
- [37] M. Geva, R. Schuster, J. Berant, and O. Levy, "Transformer feed-forward layers are key-value memories," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 5484–5495.
- [38] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen *et al.*, "Lora: Low-rank adaptation of large language models," *Iclr*, vol. 1, no. 2, p. 3, 2022.
- [39] S. Welleck, I. Kulikov, S. Roller, E. Dinan, K. Cho, and J. Weston, "Neural text generation with unlikelihood training," *arXiv preprint arXiv:1908.04319*, 2019.
- [40] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. Li *et al.*, "Deepseek-coder: when the large language model meets programming—the rise of code intelligence," *arXiv preprint arXiv:2401.14196*, 2024.
- [41] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.
- [42] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

- [43] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM computing surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [44] A. Alansari and H. Luqman, "Large language models hallucination: A comprehensive survey," *Computer Science Review*, vol. 61, p. 100970, 2026.
- [45] F. Liu, Y. Liu, L. Shi, H. Huang, R. Wang, Z. Yang, L. Zhang, Z. Li, and Y. Ma, "Exploring and evaluating hallucinations in llm-powered code generation," *arXiv e-prints*, pp. arXiv–2404, 2024.
- [46] Y. Tian, W. Yan, Q. Yang, X. Zhao, Q. Chen, W. Wang, Z. Luo, L. Ma, and D. Song, "Codehalu: Investigating code hallucinations in llms via execution-based verification," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 24, 2025, pp. 25 300–25 308.
- [47] S. Zhang, T. Yu, and Y. Feng, "Truthx: Alleviating hallucinations by editing large language models in truthful space," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 8908–8949.
- [48] A. Chen, P. Pasupat, S. Singh, H. Lee, and K. Guu, "Purr: Efficiently editing language model hallucinations by denoising language model corruptions," *arXiv preprint arXiv:2305.14908*, 2023.
- [49] A. Sinitsin, V. Plokhotnyuk, D. Pyrkun, S. Popov, and A. Babenko, "Editable neural networks," *arXiv preprint arXiv:2004.00345*, 2020.
- [50] C. Zhu, A. S. Rawat, M. Zaheer, S. Bhojanapalli, D. Li, F. Yu, and S. Kumar, "Modifying memories in transformer models," *arXiv preprint arXiv:2012.00363*, 2020.
- [51] P. Wang, N. Zhang, B. Tian, Z. Xi, Y. Yao, Z. Xu, M. Wang, S. Mao, X. Wang, S. Cheng *et al.*, "Easyedit: An easy-to-use knowledge editing framework for large language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, 2024, pp. 82–93.
- [52] E. Mitchell, C. Lin, A. Bosselut, C. D. Manning, and C. Finn, "Memory-based model editing at scale," in *International Conference on Machine Learning*. PMLR, 2022, pp. 15 817–15 831.
- [53] T. Hartvigsen, S. Sankaranarayanan, H. Palangi, Y. Kim, and M. Ghassemi, "Aging with grace: Lifelong model editing with discrete key-value adaptors," *Advances in Neural Information Processing Systems*, vol. 36, pp. 47 934–47 959, 2023.
- [54] C. Zheng, L. Li, Q. Dong, Y. Fan, Z. Wu, J. Xu, and B. Chang, "Can we edit factual knowledge by in-context learning?" in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 4862–4876.
- [55] N. De Cao, W. Aziz, and I. Titov, "Editing factual knowledge in language models," in *Proceedings of the 2021 conference on empirical methods in natural language processing*, 2021, pp. 6491–6506.
- [56] E. Mitchell, C. Lin, A. Bosselut, C. Finn, and C. D. Manning, "Fast model editing at scale," *arXiv preprint arXiv:2110.11309*, 2021.
- [57] J.-C. Gu, H.-X. Xu, J.-Y. Ma, P. Lu, Z.-H. Ling, K.-W. Chang, and N. Peng, "Model editing harms general abilities of large language models: Regularization to the rescue," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 16 801–16 819.
- [58] A. Gupta, A. Rao, and G. Anumanchipalli, "Model editing at scale leads to gradual and catastrophic forgetting," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 15 202–15 232.
- [59] anonymous, "Replication package," 2026. [Online]. Available: <https://doi.org/10.5281/zenodo.21066848>
- [60] "Replication package," 2026. [Online]. Available: <https://anonymous.4open.science/r/BOUND-BA7D/>